16B

CSS LAYOUT WITH GRID

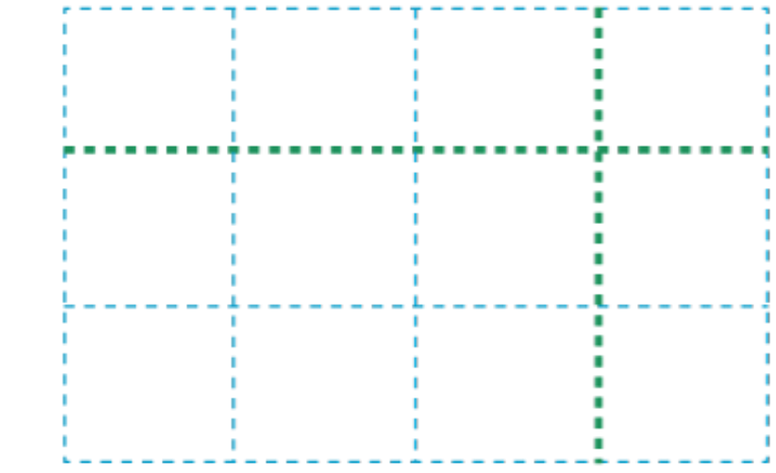
How CSS Grids Work

1. Set an element's **`display`** to **`grid`** to establish a **grid container**. Its children become **grid items**.

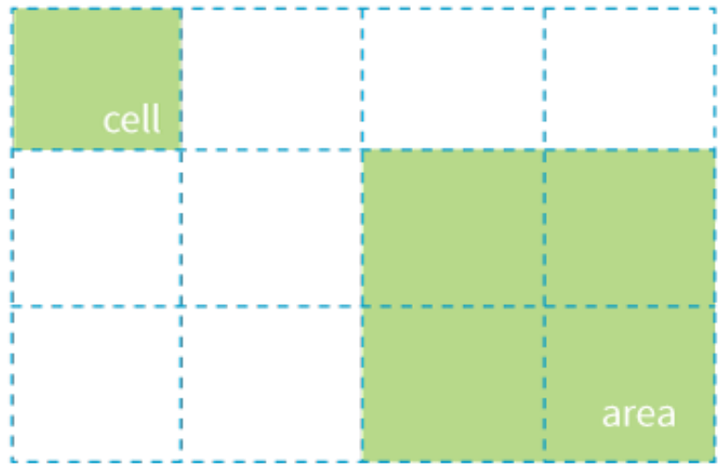
2. Set up the columns and rows for the grid (explicitly or with rules for how they are created on the fly).

3. Assign each grid item to an area on the grid (or let them flow in automatically in sequential order).

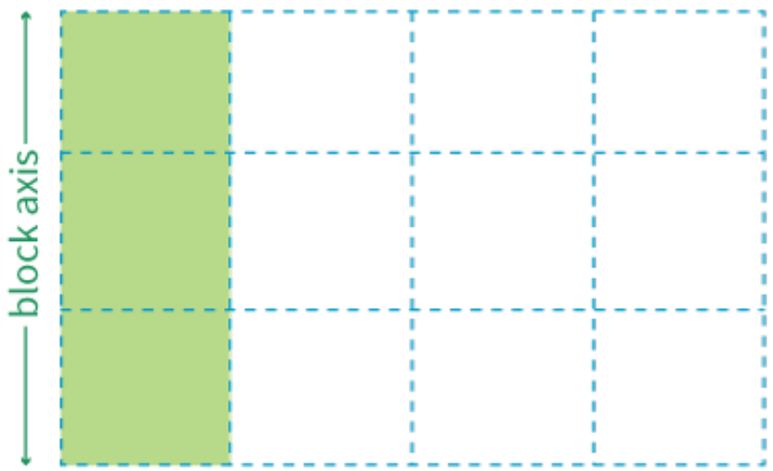
Grid Terminology



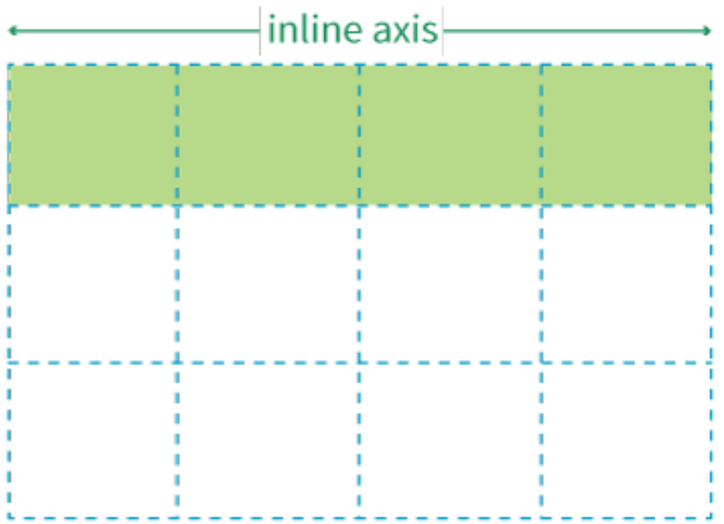
grid lines



grid cell and grid area



grid track (column)



grid track (row)

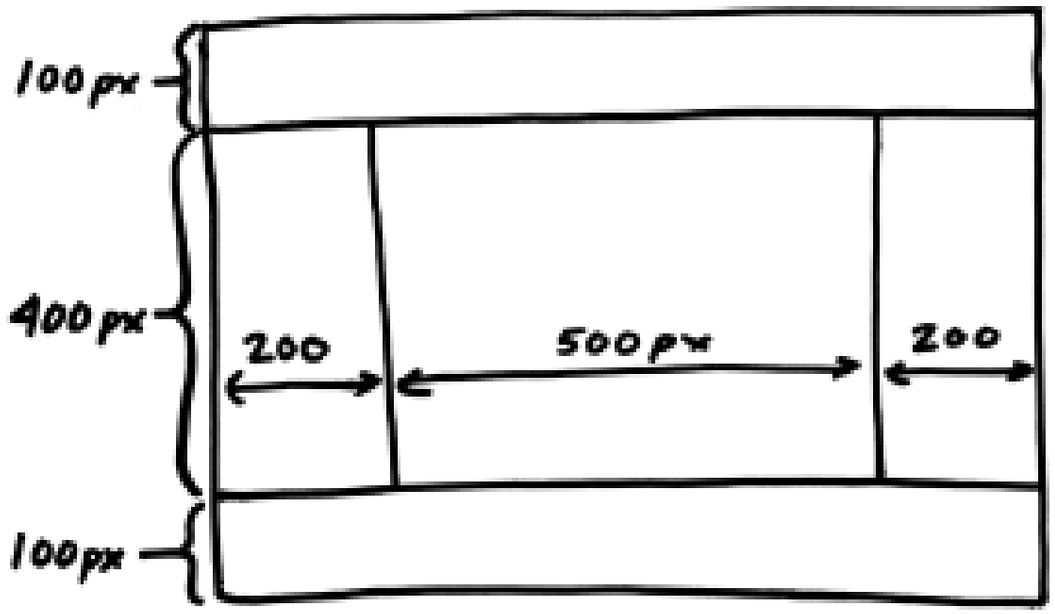


FIGURE 16-31. A rough sketch for my grid-based page layout. The dotted lines in the bottom image show how many rows and columns the grid requires to create the layout structure.

Creating a Grid Container

To make an element a **grid container**, set its **`display`** property to **grid**.

All of its children automatically become **grid items**.

The markup

```
<div id="layout">
  <div id="one">One</div>
  <div id="two">Two</div>
  <div id="three">Three</div>
  <div id="four">Four</div>
  <div id="five">Five</div>
</div>
```

The styles

```
#layout {
  display: grid;
}
```

Defining Row and Column Tracks

`grid-template-rows`
`grid-template-columns`

Values: `none`, *list of track sizes and optional line names*

- The value of **`grid-tempate-rows`** is a list of the *heights* of each row track in the grid.
- The value of **`grid-template-columns`** is a list of the *widths* of each column track in the grid.

```
#layout {
  display: grid;
  grid-template-rows: 100px 400px 100px;
  grid-template-columns: 200px 500px 200px;
}
```

- The number of sizes provided determines the number of rows/columns in the grid. This grid in the example above has 3 rows and 3 columns.

```
#layout {
  display: grid;
  grid-template-rows: 100px 400px 100px;
  grid-template-columns: 200px 500px 200px;
}
```

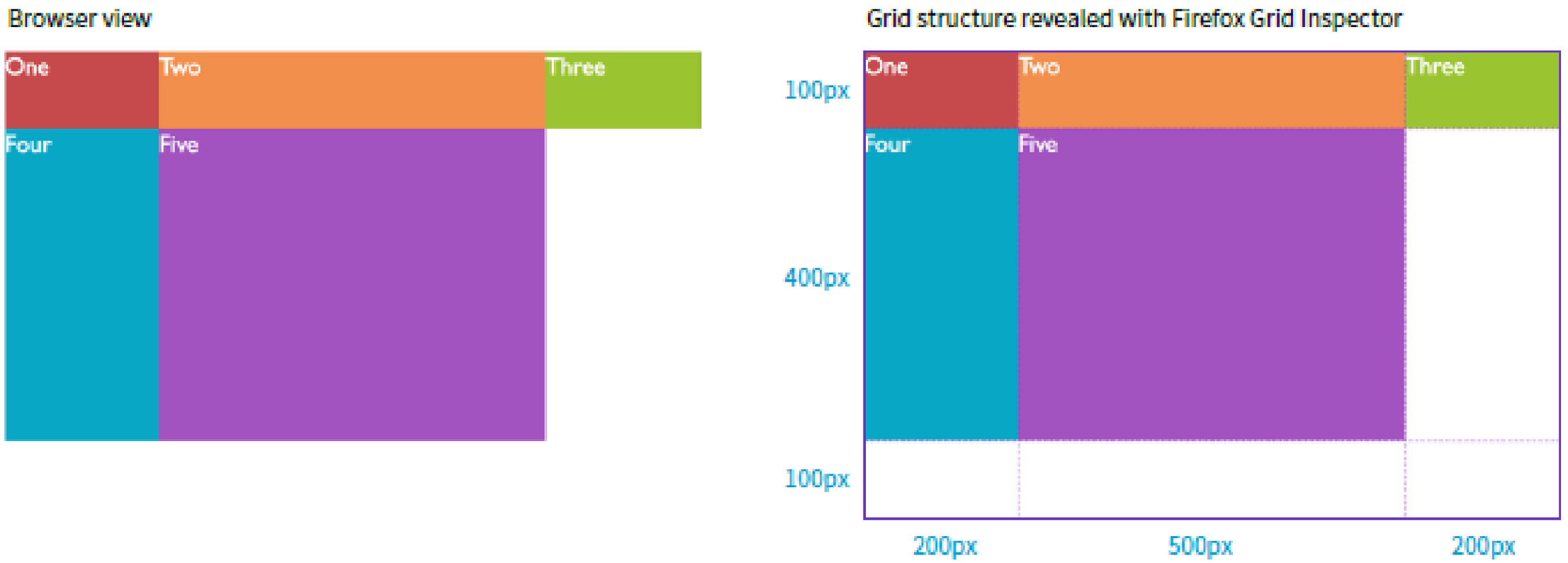


FIGURE 16-32. By default, grid items flow into the grid cells by rows.

Grid Line Names

You can also assign names to lines to make them more intuitive to reference later.

Grid line names are added in square brackets in the position they appear relative to the tracks.

To give a line more than one name, include all the names in brackets, separated by spaces.

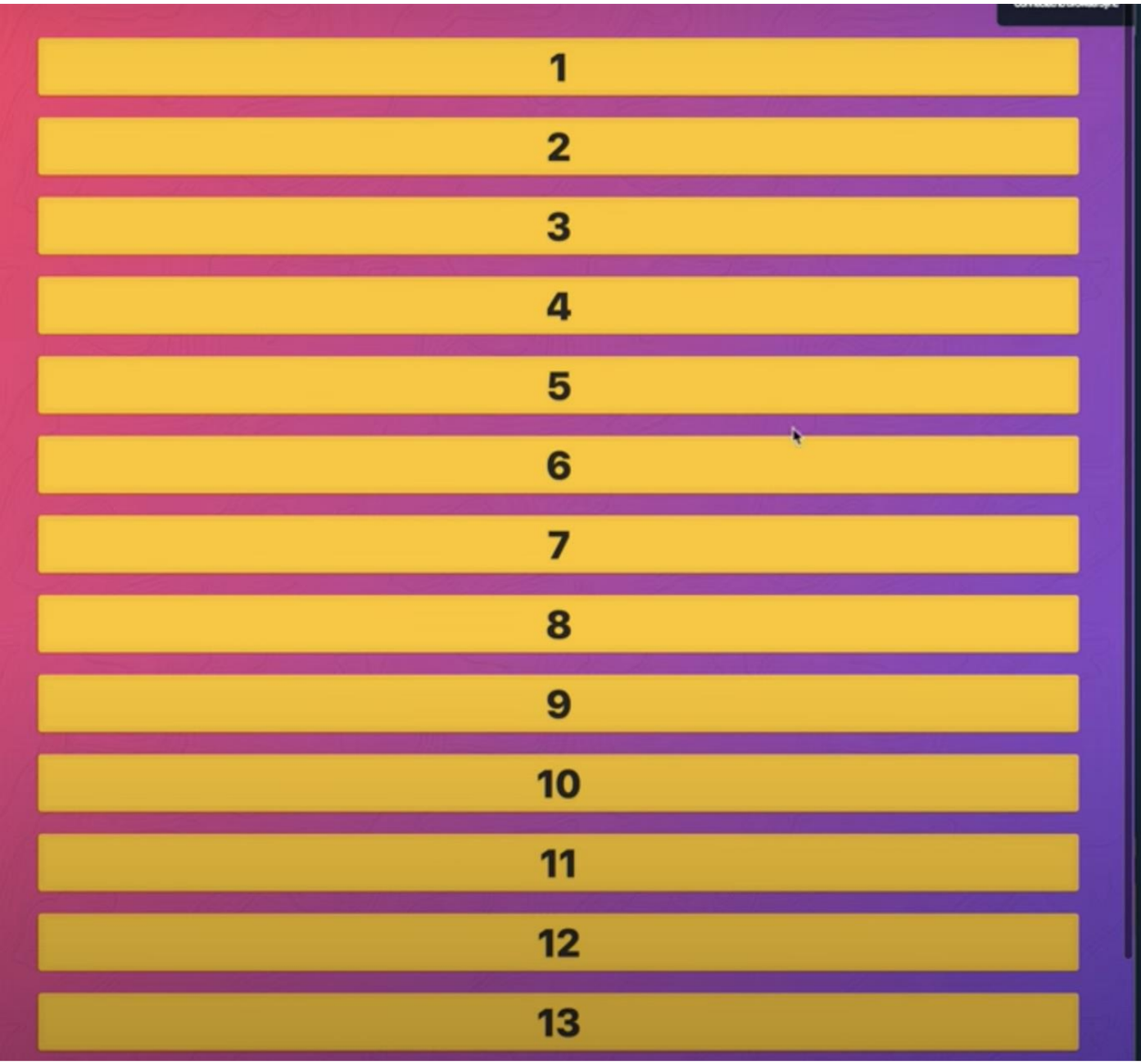
```
#layout {
  display: grid;
  grid-template-rows: [header-start] 100px [header-end
content-start] 400px [content-end footer-start] 100px;
  grid-template-columns: [ads] 200px [main] 500px [links]
200px;
}
```

Track Size Values

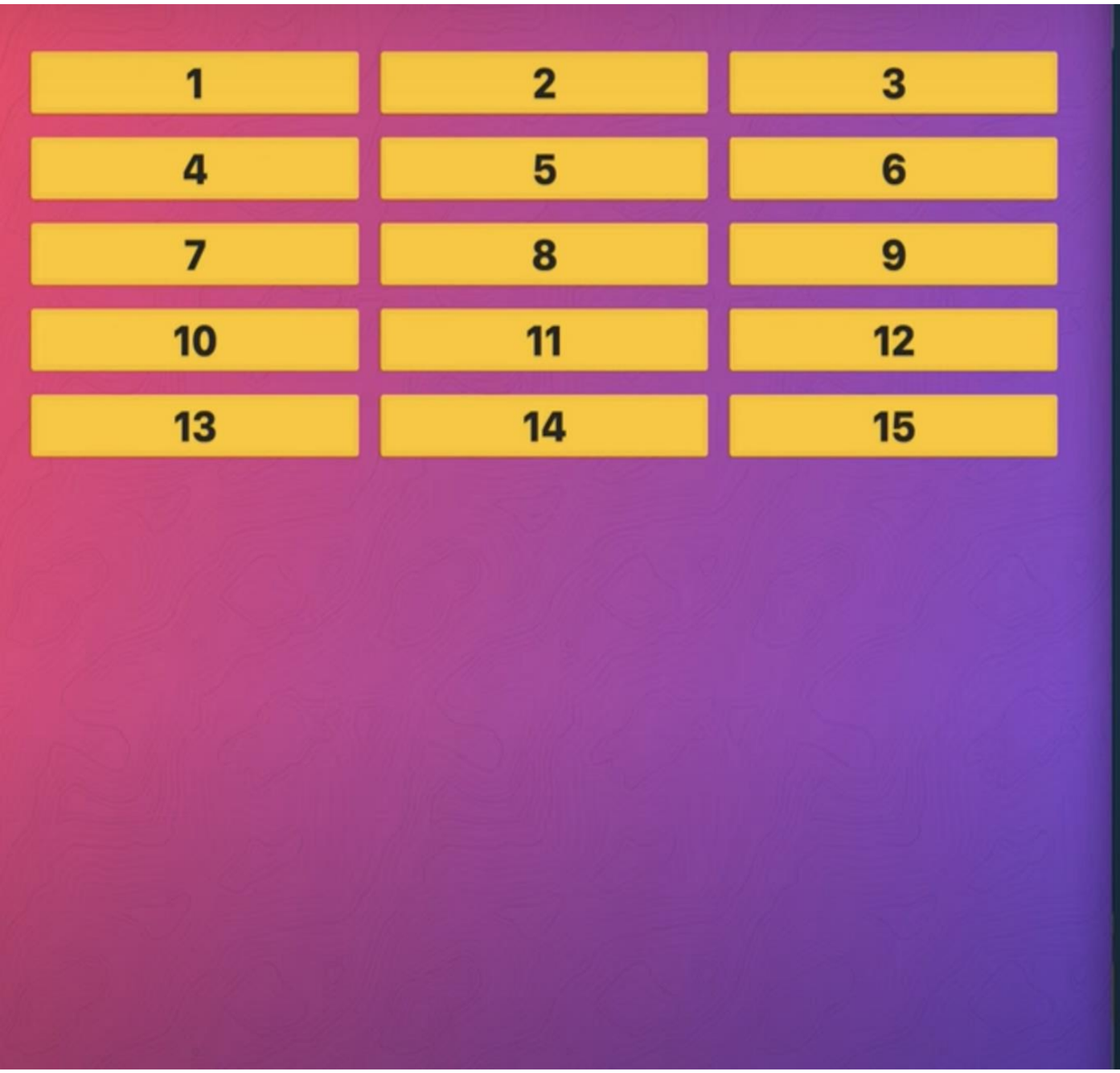
The CSS Grid spec provides a *lot* of ways to specify the width and height of a track. Some of these ways allow tracks to adapt to available space and/or to the content they contain:

- Lengths (such as `px` or `em`)
- Percentage values (%)
- Fractional units (`fr`)
- `minmax()`
- `min-content`, `max-content`
- `auto`
- `fit-content()`

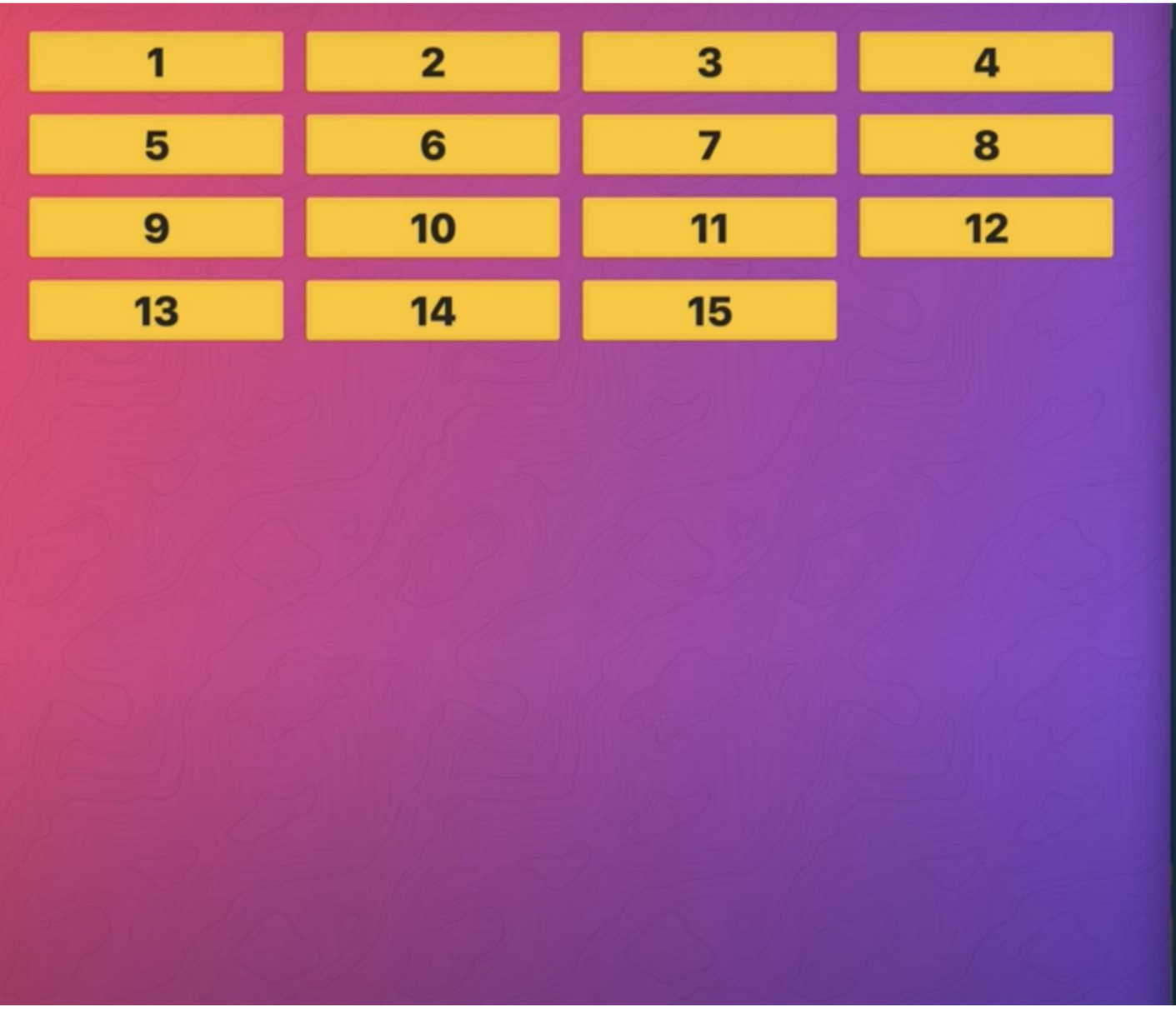
https://mozilladevelopers.github.io/playground/css-grid/04-fr-unit/



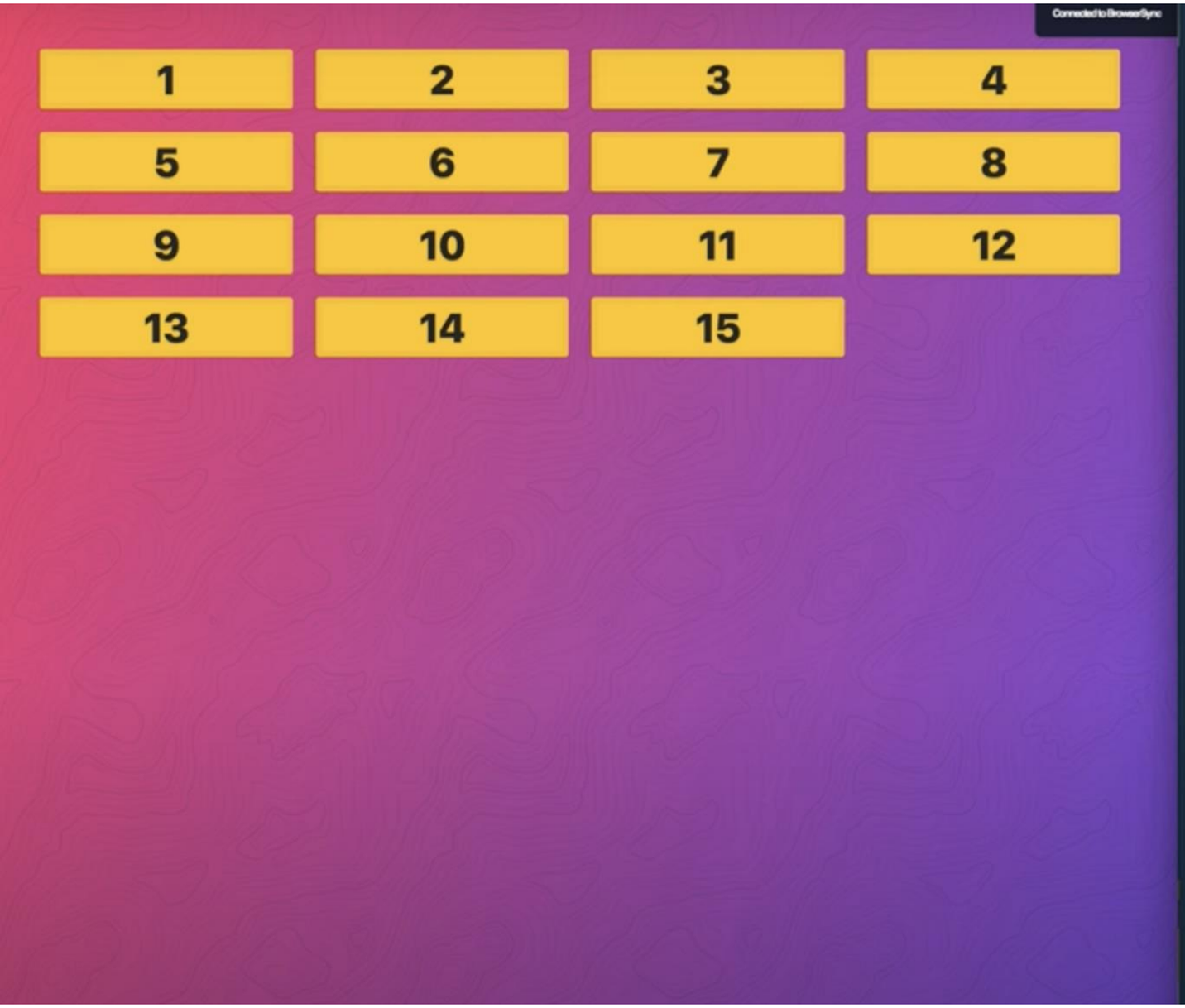

```
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
    <div class="item">7</div>
    <div class="item">8</div>
    <div class="item">9</div>
    <div class="item">10</div>
    <div class="item">11</div>
    <div class="item">12</div>
    <div class="item">13</div>
    <div class="item">14</div>
    <div class="item">15</div>
  </div>

  <style>
    .container {
      display: grid;
      grid-gap: 20px;
    }
  </style>
</body>

</html>
```



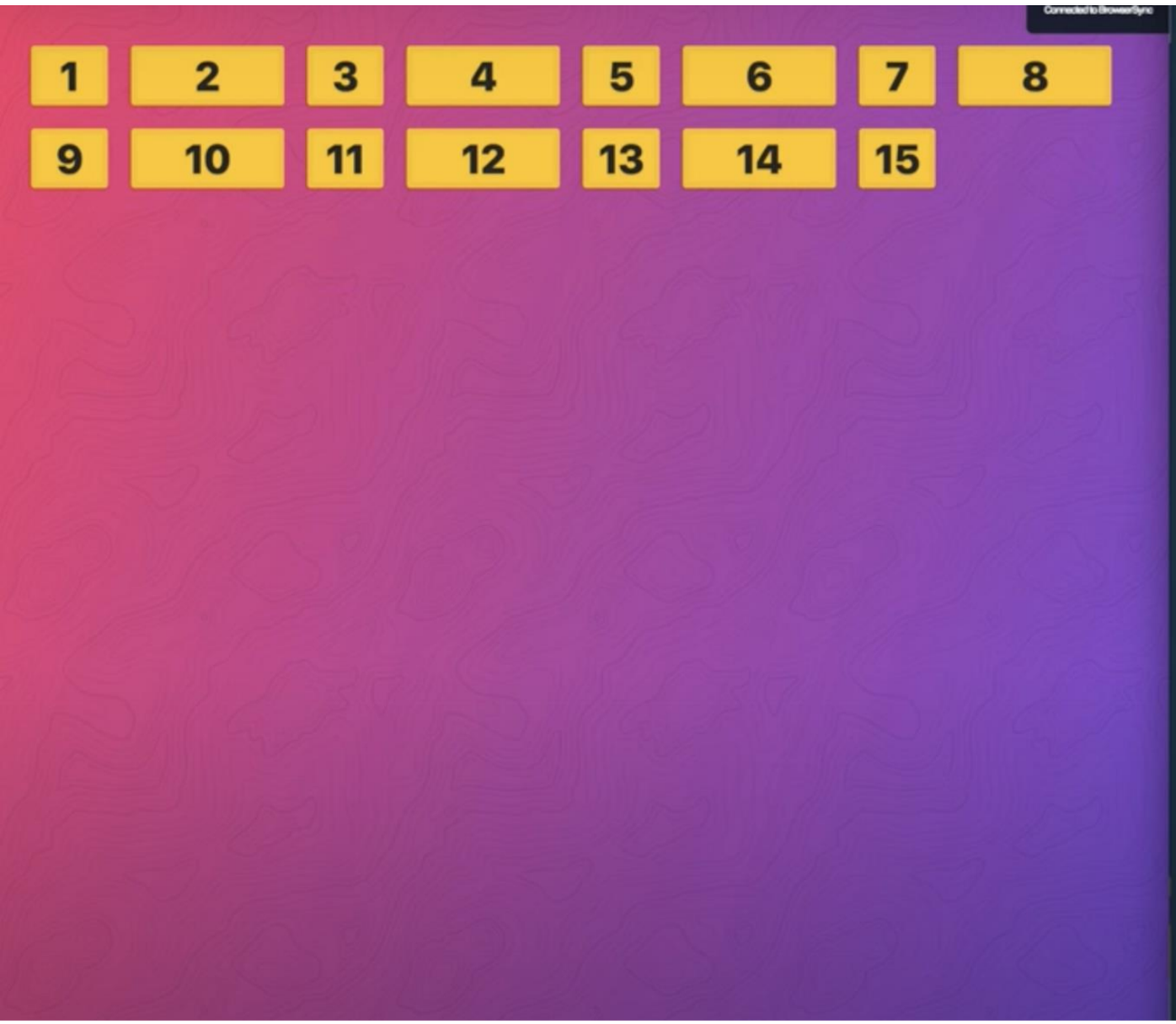

```
14      <div class="item">2</div>
15      <div class="item">3</div>
16      <div class="item">4</div>
17      <div class="item">5</div>
18      <div class="item">6</div>
19      <div class="item">7</div>
20      <div class="item">8</div>
21      <div class="item">9</div>
22      <div class="item">10</div>
23      <div class="item">11</div>
24      <div class="item">12</div>
25      <div class="item">13</div>
26      <div class="item">14</div>
27      <div class="item">15</div>
28    </div>
29
30    <style>
31      .container {
32        display: grid;
33        grid-gap: 20px;
34        grid-template-columns: 1fr 1fr 1fr;
35      }
36    </style>
37  </body>
38
39  </html>
```



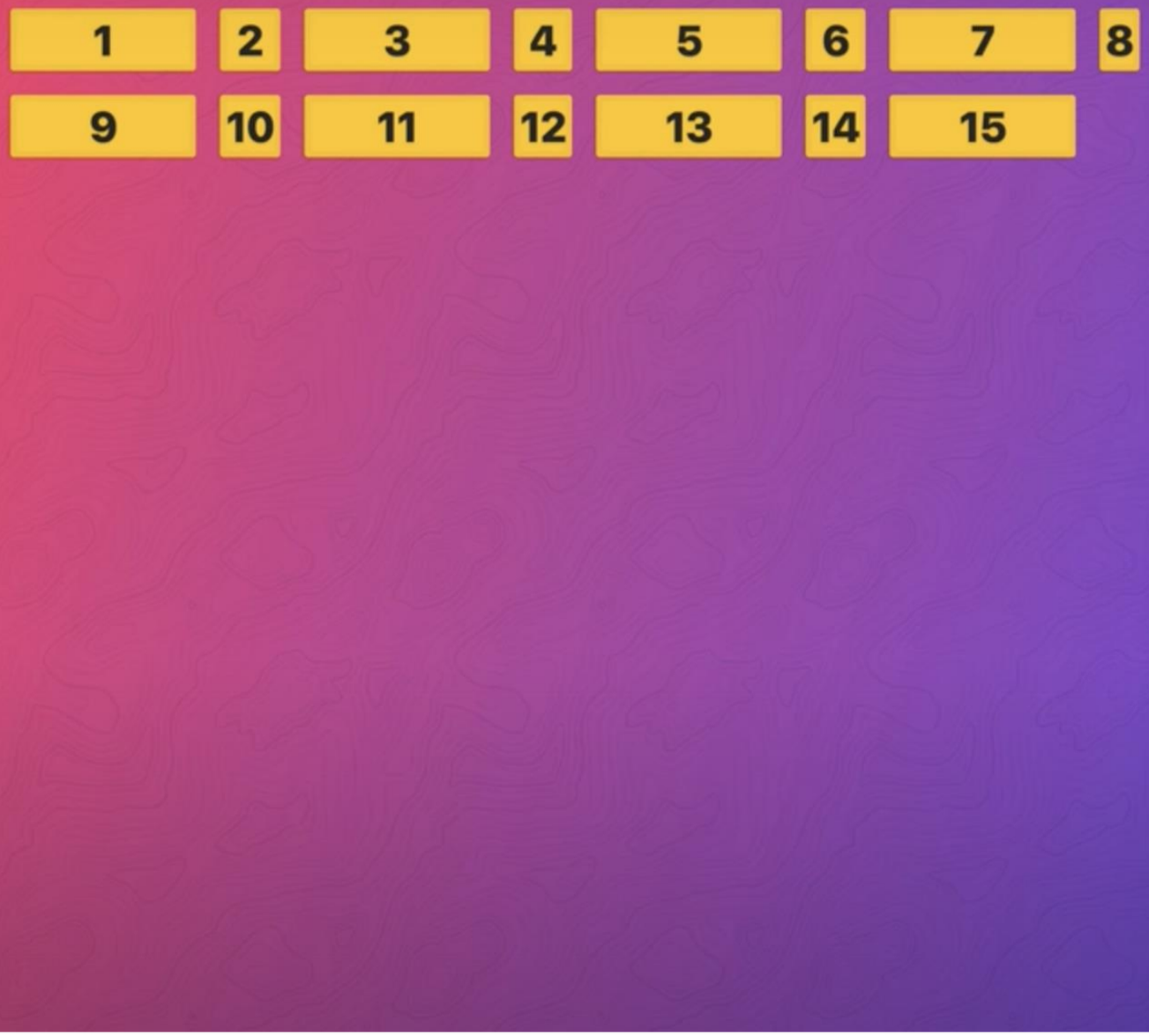

```
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
    <div class="item">7</div>
    <div class="item">8</div>
    <div class="item">9</div>
    <div class="item">10</div>
    <div class="item">11</div>
    <div class="item">12</div>
    <div class="item">13</div>
    <div class="item">14</div>
    <div class="item">15</div>
  </div>

  <style>
    .container {
      display: grid;
      grid-gap: 20px;
      grid-template-columns: 1fr 1fr 1fr 1fr;
    }
  </style>
</body>

</html>
```




```
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
    <div class="item">7</div>
    <div class="item">8</div>
    <div class="item">9</div>
    <div class="item">10</div>
    <div class="item">11</div>
    <div class="item">12</div>
    <div class="item">13</div>
    <div class="item">14</div>
    <div class="item">15</div>
  </div>

  <style>
    .container {
      display: grid;
      grid-gap: 20px;
      grid-template-columns: repeat(4, 1fr);
    }
  </style>
</body>

</html>
```




```
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
    <div class="item">7</div>
    <div class="item">8</div>
    <div class="item">9</div>
    <div class="item">10</div>
    <div class="item">11</div>
    <div class="item">12</div>
    <div class="item">13</div>
    <div class="item">14</div>
    <div class="item">15</div>
  </div>

  <style>
    .container {
      display: grid;
      grid-gap: 20px;
      grid-template-columns: repeat(4, 1fr 2fr);
    }
  </style>
</body>

</html>
```




```
    <div class="item">2</div>
    <div class="item">3</div>
    <div class="item">4</div>
    <div class="item">5</div>
    <div class="item">6</div>
    <div class="item">7</div>
    <div class="item">8</div>
    <div class="item">9</div>
    <div class="item">10</div>
    <div class="item">11</div>
    <div class="item">12</div>
    <div class="item">13</div>
    <div class="item">14</div>
    <div class="item">15</div>
  </div>

  <style>
    .container {
      display: grid;
      grid-gap: 20px;
      grid-template-columns: repeat(4, 1fr auto);
    }
  </style>
</body>

</html>
```

Spacing Between Tracks

`grid-row-gap`
`grid-column-gap`

Values: *Length (must not be negative)*

`grid-gap`

Values: *`grid-row-gap grid-column-gap`*

Adds space between the row and/or columns tracks of the grid

NOTE: These property names will be changing to `row-gap`, `column-gap`, and `gap`, but the new names are not yet supported.

Space Between Tracks (cont'd)

If you want equal space between all tracks in a grid, use a gap instead of creating additional spacing tracks:

```
grid-gap: 20px 50px;
```

(Adds 20px space between rows and 50px between columns)

Item Alignment

`justify-self`
`align-self`

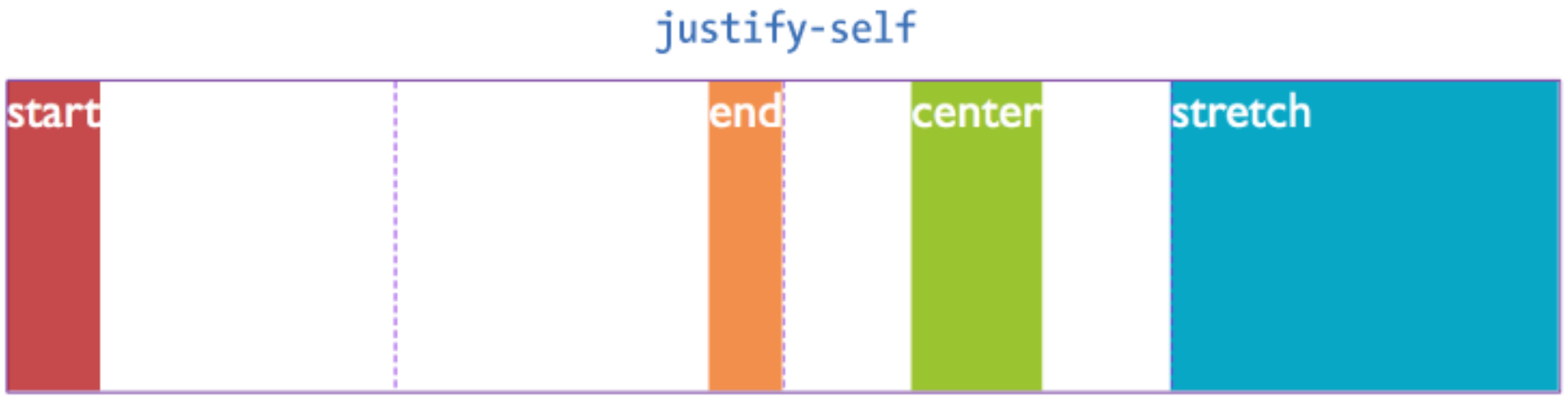
Values: `start`, `end`, `center`, `left`, `right`, `self-start`, `self-end`, `stretch`, `normal`, `auto`

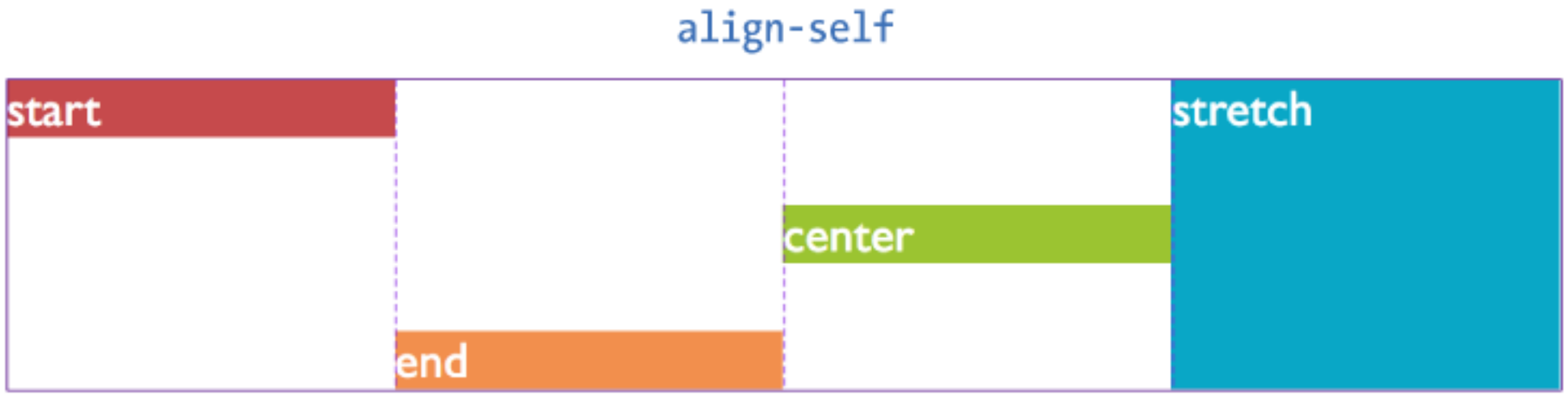
When an item element doesn't fill its grid cell, you can specify how it should be aligned within the cell.

`justify-self` aligns on the **inline** axis (horizontal for L-to-R languages).

`align-self` aligns on the **block** (vertical) axis.

Item Alignment (cont'd)





NOTE: These properties are applied to the individual **grid item** element.

Aligning All the Items

`justify-items`
`align-items`

Values: `start`, `end`, `center`, `left`, `right`, `self-start`, `self-end`, `stretch`, `normal`, `auto`

These properties align items in their cells all at once. They are applied to the **grid container**.

`justify-items` aligns on the **inline** axis.

`align-items` aligns on the **block** (vertical) axis.

Track Alignment

`justify-content`
`align-content`

Values: `start`, `end`, `center`, `left`, `right`, `stretch`, `space-around`, `space-between`, `space-evenly`

When the **grid tracks do not fill the entire container**, you can specify how tracks align.

`justify-content` aligns on the **inline** axis (horizontal for L-to-R languages).

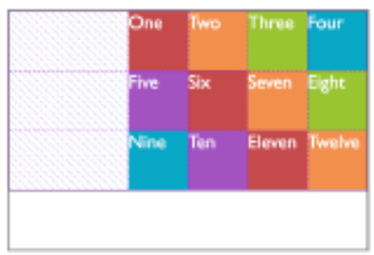
`align-content` aligns on the **block** (vertical) axis.

Track Alignment (cont'd)

justify-content:

start

end



center

space-around

space-between

space-evenly

align-content:

start

end

center

space-around

space-between

space-evenly

NOTE: These properties are applied to the **grid container**.